AMD EMBEDDED DEVELOPMENT FRAMEWORK

Migrating a Vivado design to EDF with agentic AI

Agent reads and cross-checks. Tools validate. Engineer decides.

1 · Interrogate
Real IP config and address map

2 · Rebuild PL
Scripted design → bitstream + XSA

3 · Firmware
Overlay + shell.json for dfx-mgr

4 · Image
Own layer, BitBake, inspect

5 · Bring-up
Verify each boundary

Coding agent

Reads sources, docs and bindings; drafts Tcl, overlays, recipes.

Vivado MCP

Live Tcl: query IP, validate, implement, report.

Engineer

Runs `bitbake` and the board; approves every change.

STEPS 1–2 · HARDWARE

Know the design, then rebuild it for Linux

1. Interrogate via Vivado MCP

- Extract IP config, widths, clocks, IRQs, address map
- Trust the project, not the requirements doc
- Flag what won't port: missing IP, fixed buffers

```
get_bd_addr_segs -hier
report_property [get_bd_cells -hier *]
```

2. Rebuild as a Tcl script

- Keep the original project as reference
- Prefer IP with upstream Linux drivers
- Iterate on `validate_bd_design`; fix causes

```
report_timing_summary ; report_drc
write_hw_platform -fixed -include_bit hw.xsa
```

Gate: setup/hold slack ≥ 0 · zero DRC errors · XSA matches the address map

STEP 3 · FIRMWARE BUNDLE

Describe the PL to Linux

Device-tree overlay

- Agent drafts from address map, AMD reference overlays, kernel bindings
- Check `reg`, `interrupts`, clocks per node
- Compile early

```
dtc -@ -I dts -O dtb -o app.dtbo app.dtso
```

Firmware recipe

- Bundle: `.bit.bin` + `.dtbo` + `shell.json`
- Install to `/lib/firmware/xilinx/app`
- `DEPENDS = "dtc-native"` (build)
- `RDEPENDS:${PN} = "dfx-mgr"` (target)

Artifact: one firmware package `dfx-mgr` can load

STEP 4 · IMAGE

Build on EDF; inspect before flashing

Workspace

- AMD EDF build container
- Bind-mount a host directory
- `repo` sync the EDF manifest

Your layer

- Image, app and firmware recipes
- systemd units
- Kernel `.cfg` fragment
- One consistent `MACHINE`

Inspect

- Agent checks manifest and packages
- Loop-mount the rootfs
- Confirm firmware, units, tools

```
MACHINE=kria-zynqmp-generic bitbake app-image
```

Rule: the build output is the product, so never hand-patch the image or board

STEP 5 · BRING-UP

Verify one boundary at a time

Checks

```
xmutil listapps         # PL loaded
dmesg | grep -i probe   # drivers bound
media-ctl -p            # devices
journalctl -u app       # application
```

- Test sources first, real inputs last
- Cold boot, reboot, unplug/replug

Working with the agent

- Paste real output, not summaries
- One bounded check at a time
- Diagnose on board, fix in recipes
- Ask what consumes each setting
- Tools have the final say

Done: a clean build boots on fresh media and works end-to-end with no manual steps